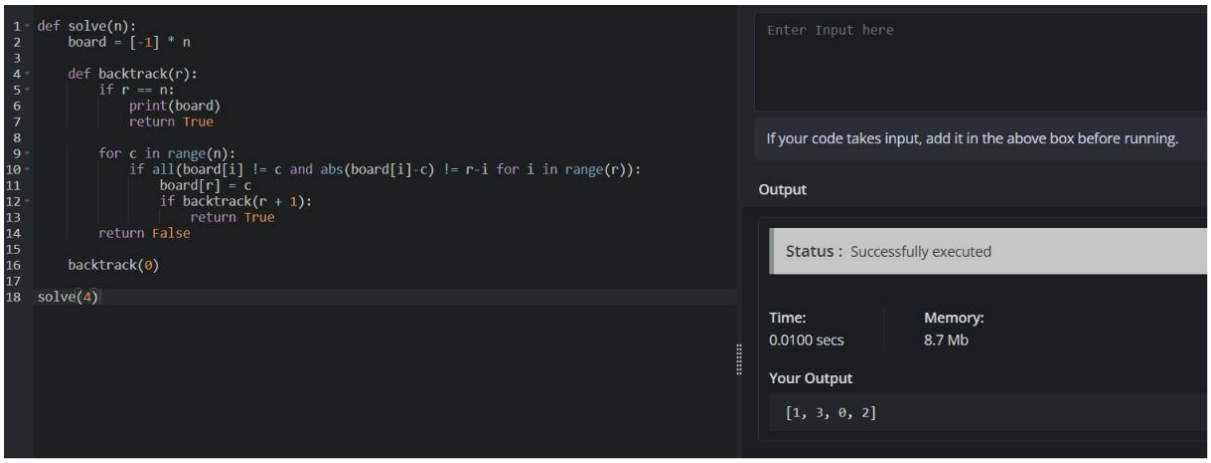
UNIT – 6 LAB EXERCISE

NAME : Jeeva

REG NO : 192421334

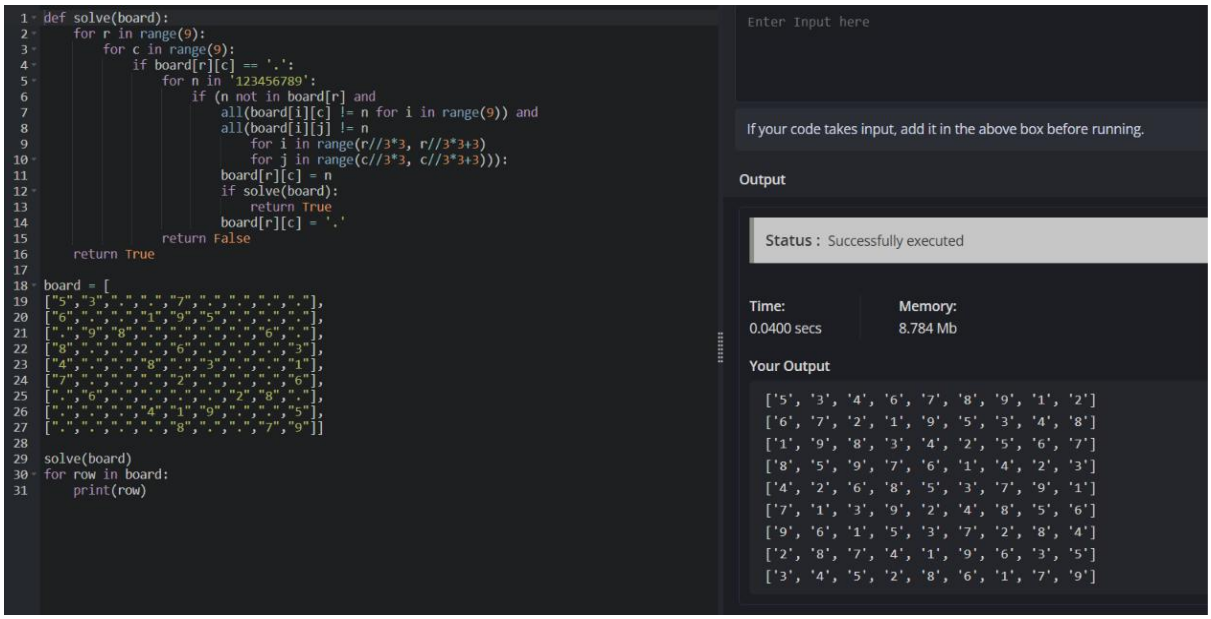
TOPIC 6 : BACKTRACKING

1. Discuss the importance of visualizing the solutions of the N-Queens Problem to understand the placement of queens better. Use a graphical representation to show how queens are placed on the board for different values of N. Explain how visual tools can help in debugging the algorithm and gaining insights into the problem's complexity. Provide examples of visual representations for N = 4, N = 5, and N = 8, showing different valid solutions.

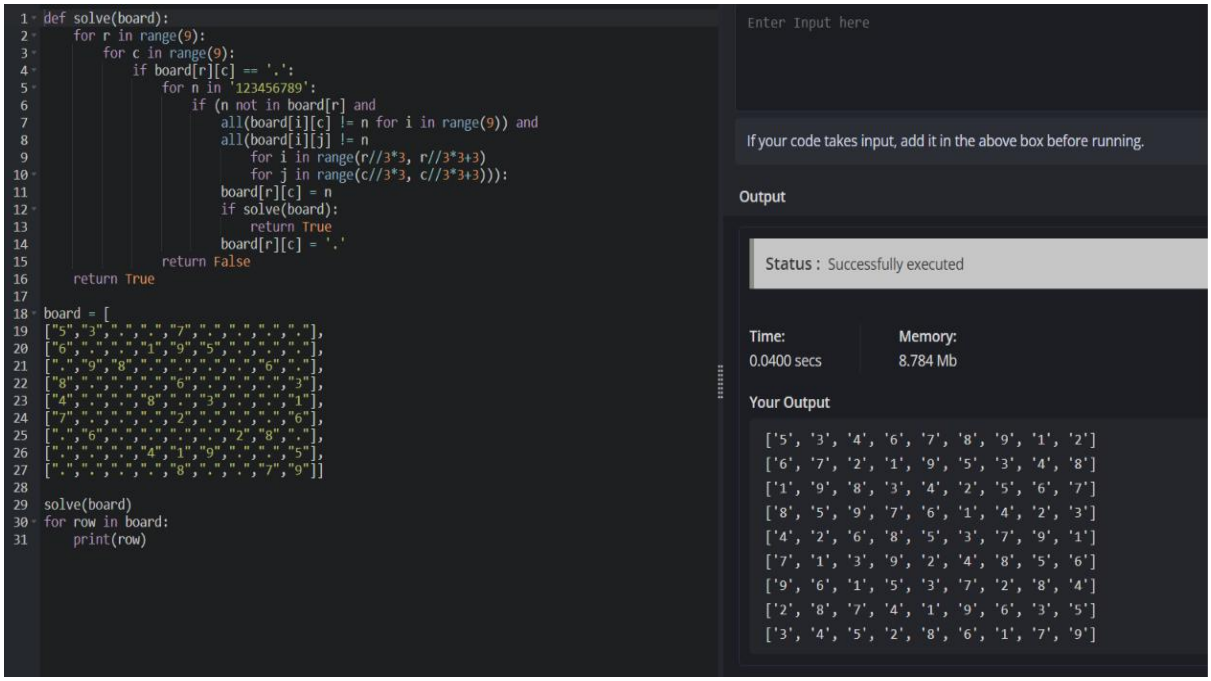


2. Discuss the generalization of the N-Queens Problem to other board sizes and shapes, such as rectangular boards or boards with obstacles. Explain how the algorithm can be adapted to handle these variations and the additional constraints they introduce. Provide examples of solving generalized N-Queens Problems for different board configurations, such as an 8×10 board, a 5×5 board with obstacles, and a 6×6 board with restricted positions.

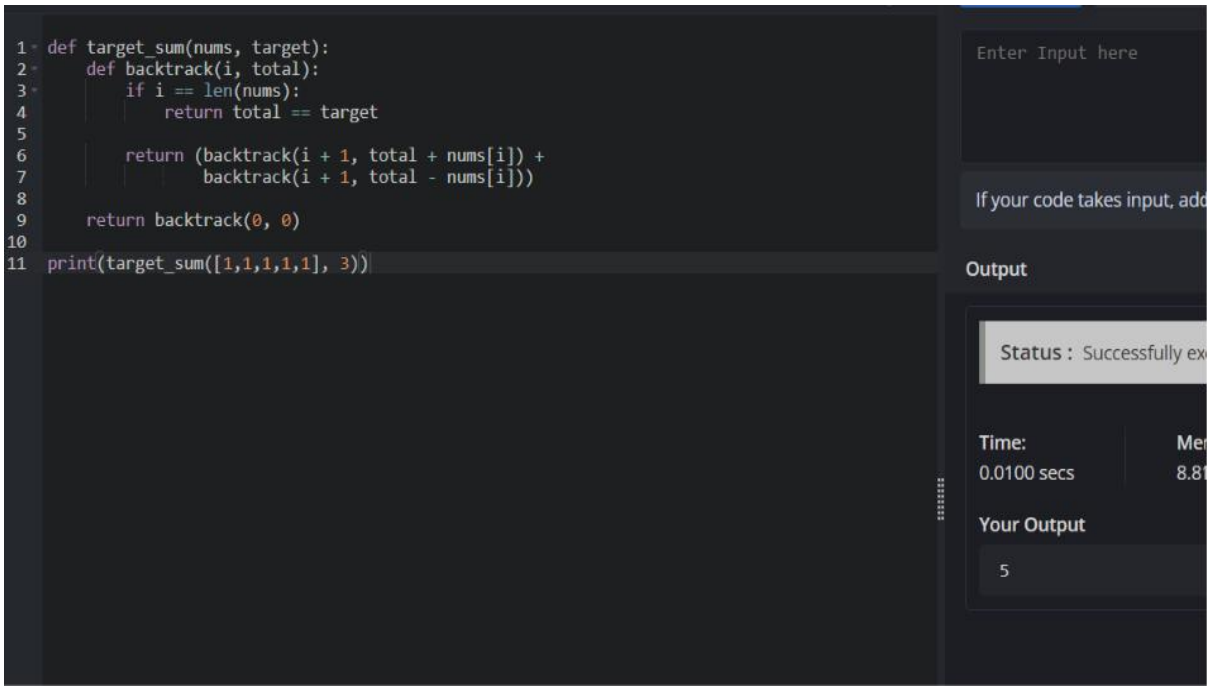
3. Write a program to solve a Sudoku puzzle by filling the empty cells.A sudoku solution must satisfy all of the following rules:Each of the digits 1-9 must occur exactly once in each row.Each of the digits 1-9 must occur exactly once in each column.Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub- boxes of the grid.The '.' character indicates empty cells.



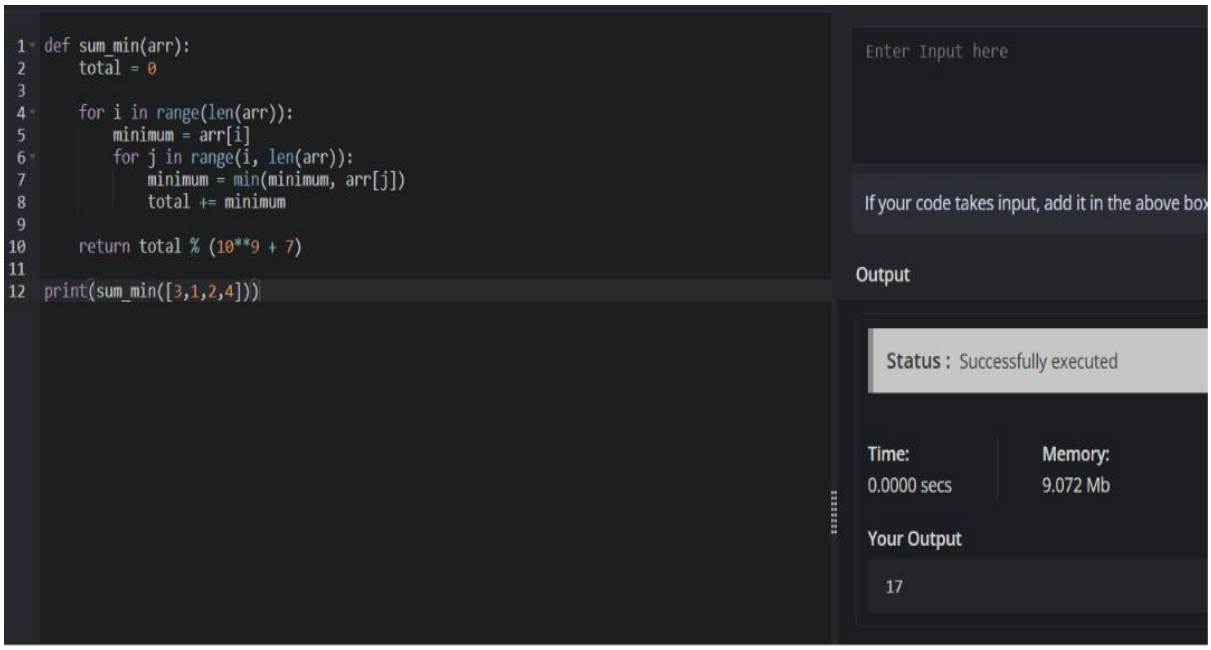
4. Write a program to solve a Sudoku puzzle by filling the empty cells.A sudoku solution must satisfy all of the following rules:Each of the digits 1-9 must occur exactly once in each row.Each of the digits 1-9 must occur exactly once in each column.Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub- boxes of the grid.The '.' character indicates empty cells.



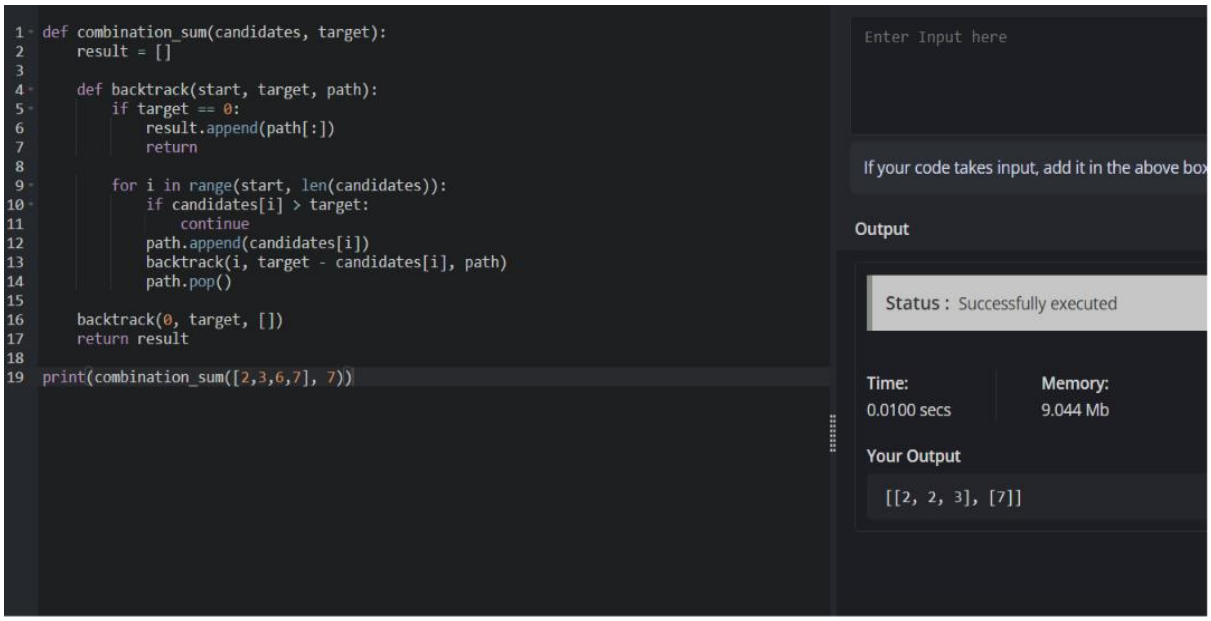
5. You are given an integer array nums and an integer target. You want to build an expression out of nums by adding one of the symbols '+' and '-' before each integer in nums and then concatenate all the integers.For example, if nums = [2, 1], you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1" Return the number of different expressions that you can build, which evaluates to target.



6. Given an array of integers arr, find the sum of min(b), where b ranges over every (contiguous) subarray of arr. Since the answer may be large, return the answer modulo 109 + 7.



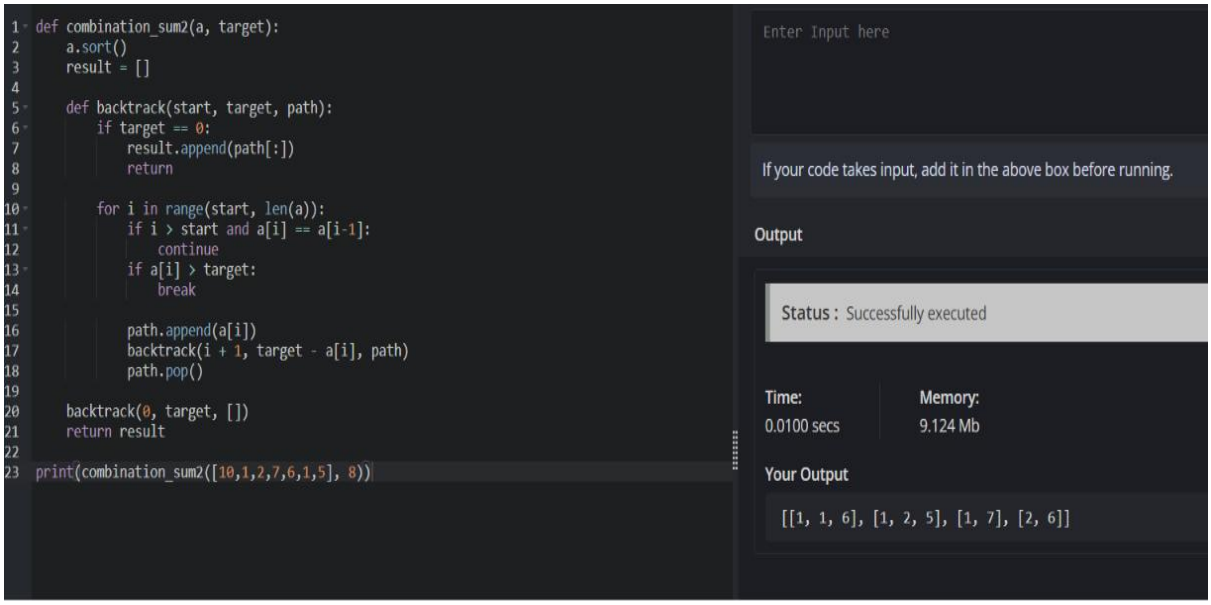
7. Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.



```python
def combination_sum(candidates, target):
    result = []

    def backtrack(start, target, path):
        if target == 0:
            result.append(path[:])
            return

        for i in range(start, len(candidates)):
            if candidates[i] > target:
                continue
            path.append(candidates[i])
            backtrack(i, target - candidates[i], path)
            path.pop()

    backtrack(0, target, [])
    return result

print(combination_sum([2,3,6,7], 7))
```

```
Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.044 Mb

Your Output

[[2, 2, 3], [7]]
```

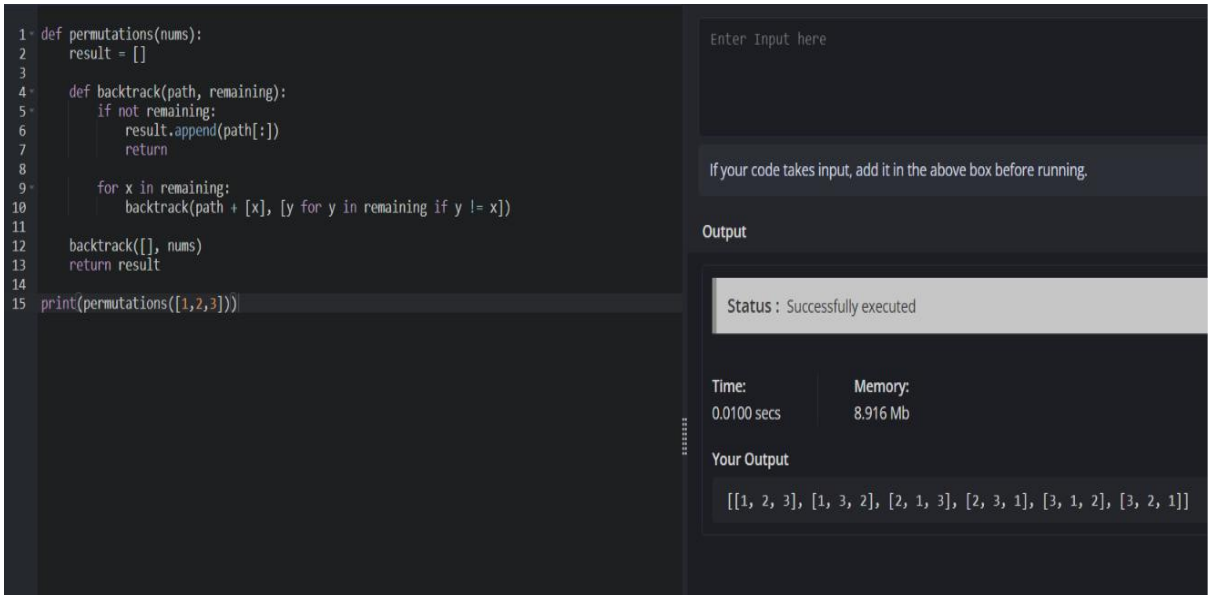
8. Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination. The solution set must not contain duplicate combinations.



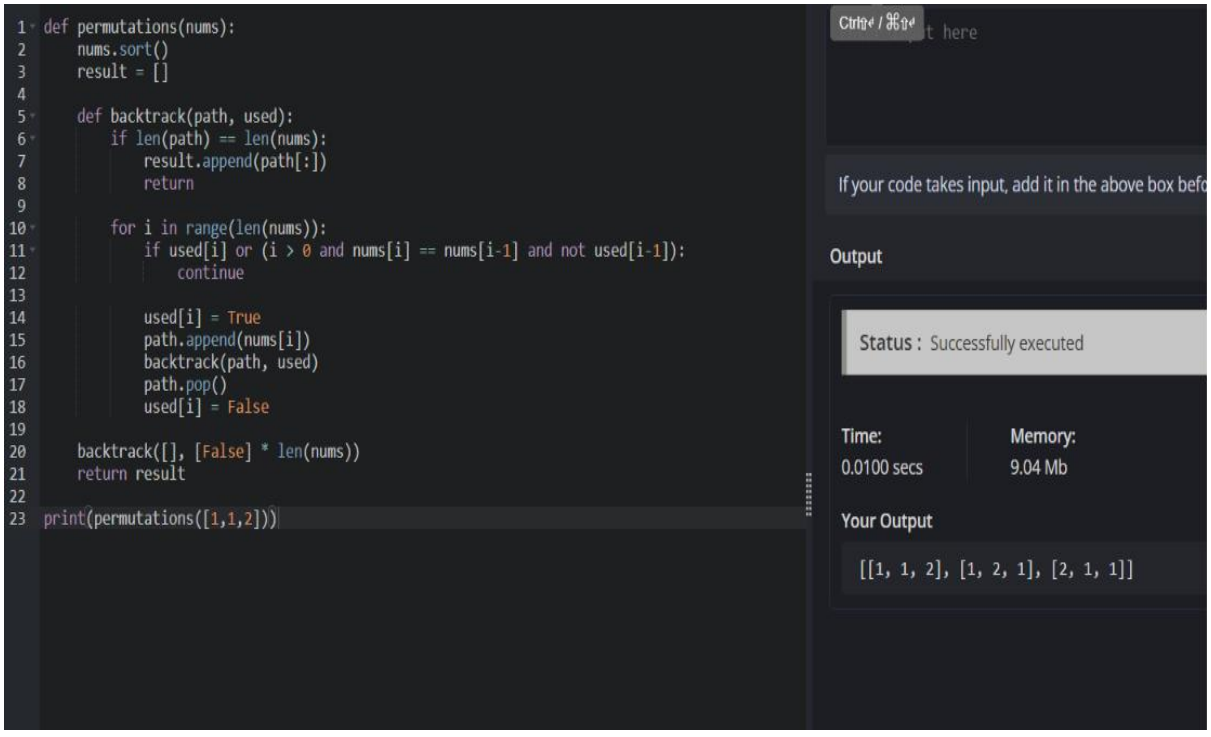
```python
def combination_sum2(a, target):
    a.sort()
    result = []

    def backtrack(start, target, path):
        if target == 0:
            result.append(path[:])
            return

        for i in range(start, len(a)):
            if i > start and a[i] == a[i-1]:
                continue
            if a[i] > target:
                break

            path.append(a[i])
            backtrack(i + 1, target - a[i], path)
            path.pop()

    backtrack(0, target, [])
    return result

print(combination_sum2([10,1,2,7,6,1,5], 8))
```

```
Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
9.124 Mb

Your Output

[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
```

9. Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.



10. Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order.



11. You and your friends are assigned the task of coloring a map with a limited number of colors. The map is represented as a list of regions and their adjacency relationships. The rules are as follows: At each step, you can choose any uncolored region and color it with any available color. Your friend Alice follows the same strategy immediately after you, and then your friend Bob follows suit. You want to maximize the number of regions you personally

color. Write a function that takes the map's adjacency list representation and returns the maximum number of regions you can color before all regions are colored. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4

```python
def graph_coloring(n, edges, k):
    graph = [[] for _ in range(n)]

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    color = [0] * n

    def valid(v, c):
        return all(color[x] != c for x in graph[v])

    def backtrack(v):
        if v == n:
            return True

        for c in range(1, k + 1):
            if valid(v, c):
                color[v] = c
                if backtrack(v + 1):
                    return True
                color[v] = 0

        return False

    return backtrack(0), color

edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]
print(graph_coloring(4, edges, 3))
```

```
Ctrl⇧↵ / ⌘⇧↵ t here

If your code takes input, add it in the above b

Output

Status : Successfully executed

Time:
0.0200 secs

Memory:
8.708 Mb

Your Output

(True, [1, 2, 3, 2])
```

12. You and your friends are tasked with coloring a map using a limited set of colors, with the following rules: At each step, you can choose any region of the map that hasn't been colored yet and color it with any available color. Your friend Alice will then color the next region using the same strategy, followed by your friend Bob. You aim to maximize the number of regions you color. Given a map represented as a list of regions and their adjacency relationships, write a function to determine the maximum number of regions you can color. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4, k = 3

```python
def graph_coloring(n, edges, k):
    graph = [[] for _ in range(n)]

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    color = [0] * n

    def valid(v, c):
        for x in graph[v]:
            if color[x] == c:
                return False
        return True

    def solve(v):
        if v == n:
            return True

        for c in range(1, k + 1):
            if valid(v, c):
                color[v] = c
                if solve(v + 1):
                    return True
                color[v] = 0

        return False

    if solve(0):
        return color
    return None


edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
print(graph_coloring(4, edges, 3))
```

```
Ctrl⇧↵ / ⌘⇧↵ t here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.704 Mb

Your Output

[1, 2, 3, 2]
```

13. You are given an undirected graph represented by a list of edges and the number of vertices n. Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: Given edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)] and n = 5

```python
def hamiltonian(n, edges):
    graph = [[] for _ in range(n)]

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    path = [0]

    def backtrack():
        if len(path) == n:
            return path[0] in graph[path[-1]]

        for v in range(1, n):
            if v not in path and v in graph[path[-1]]:
                path.append(v)
                if backtrack():
                    return True
                path.pop()

        return False

    return backtrack()

edges = [(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)]
print(hamiltonian(5, edges))
```

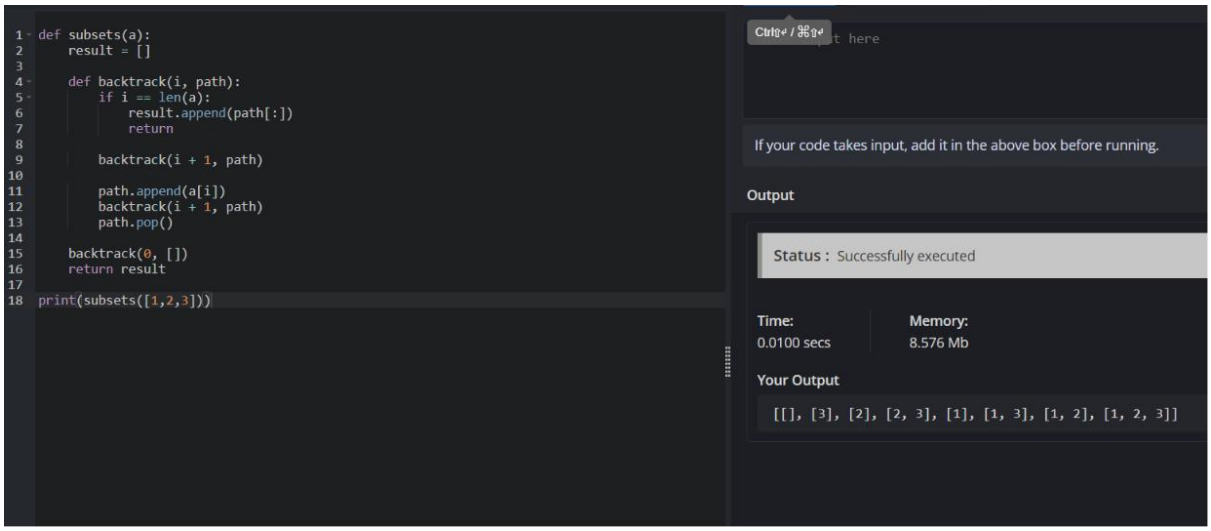
```
Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time:
0.0200 secs

Memory:
8.872 Mb

Your Output

False
```

14. You are given an undirected graph represented by a list of edges and the number of vertices n. Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example:edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] and n = 4

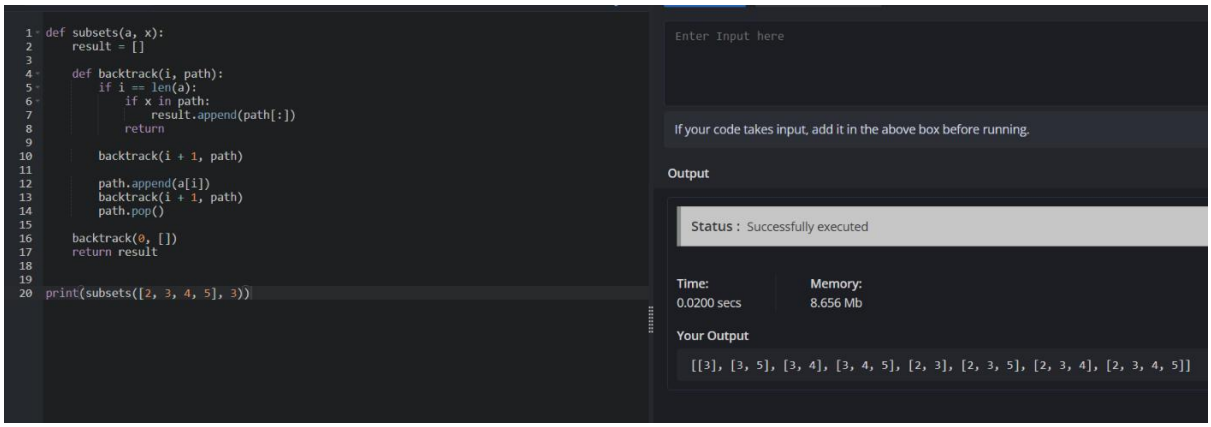
```python
def hamiltonian(n, edges):
    graph = [[] for _ in range(n)]

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    path = [0]

    def solve():
        if len(path) == n:
            return path[0] in graph[path[-1]]

        for v in range(1, n):
            if v not in path and v in graph[path[-1]]:
                path.append(v)

                if solve():
                    return True

                path.pop()

        return False

    if solve():
        return path + [0]
    return None


edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
print(hamiltonian(4, edges))
```

```
Ctrl+/ ⌘+/  t here

If your code takes input, add it in the above

Output

Status : Successfully executed

Time:
0.0100 secs

Memory:
8.792 Mb

Your Output

[0, 1, 2, 3, 0]
```

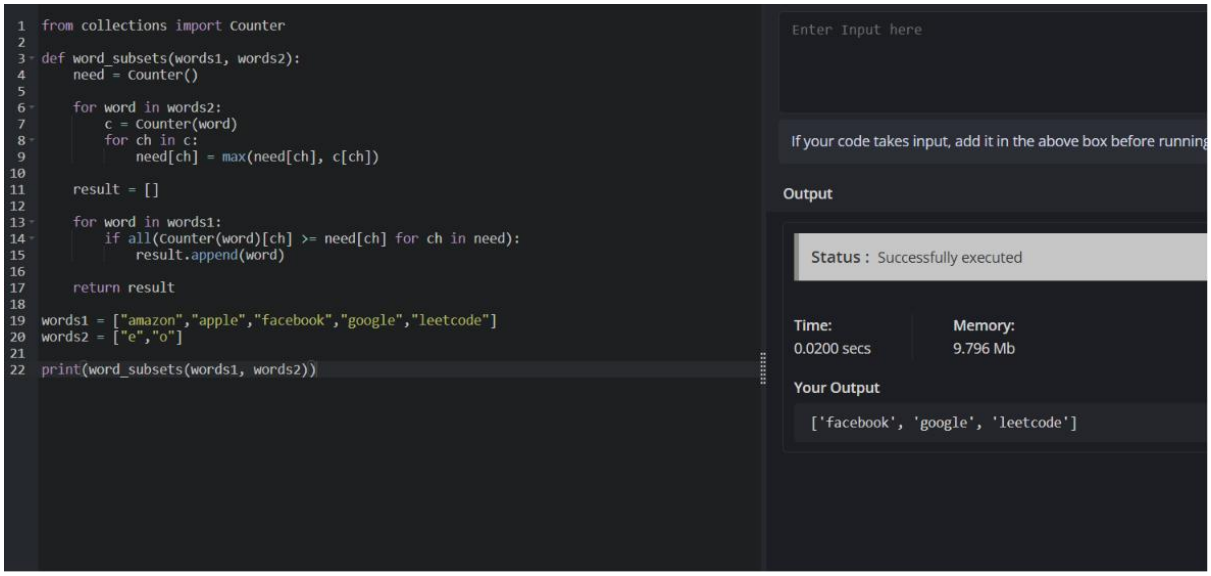
15. You are tasked with designing an efficient coading to generate all subsets of a given set S containing n elements. Each subset should be outputted in lexicographical order. Return a list of lists where each inner list is a subset of the given set. Additionally, find out how your coading handles duplicate elements in S. A = [1, 2, 3] The subsets of [1, 2, 3] are: [], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]



16. Write a program to implement the concept of subset generation. Given a set of unique integers and a specific integer 3, generate all subsets that contain the element 3. Return a list of lists where each inner list is a subset containing the element 3 E = [2, 3, 4, 5], x = 3, The subsets containing 3 : [3], [2, 3], [3, 4], [3,5], [2, 3, 4], [2, 3, 5], [3, 4, 5], [2, 3, 4, 5] Given an integer array nums of unique elements, return all possible subsets(the power set). The solution set must not contain duplicate subsets. Return the solution in any order.



17. You are given two string arrays words1 and words2. A string b is a subset of string a if every letter in b occurs in a including multiplicity. For example, "wrr" is a subset of "warrior" but is not a subset of "world". A string a from words1 is universal if for every string b in words2, b is a subset of a. Return an array of all the universal strings in words1. You may return the answer in any order.



```python
from collections import Counter

def word_subsets(words1, words2):
    need = Counter()

    for word in words2:
        c = Counter(word)
        for ch in c:
            need[ch] = max(need[ch], c[ch])

    result = []

    for word in words1:
        if all(Counter(word)[ch] >= need[ch] for ch in need):
            result.append(word)

    return result

words1 = ["amazon","apple","facebook","google","leetcode"]
words2 = ["e","o"]

print(word_subsets(words1, words2))
```